

On-policy methods: Actor-Critic

Daniil Zelezetsky

Higher School of Economics

April 21, 2025

ЧТО ИЗУЧИЛИ НА ПРОШЛОЙ ЛЕКЦИИ

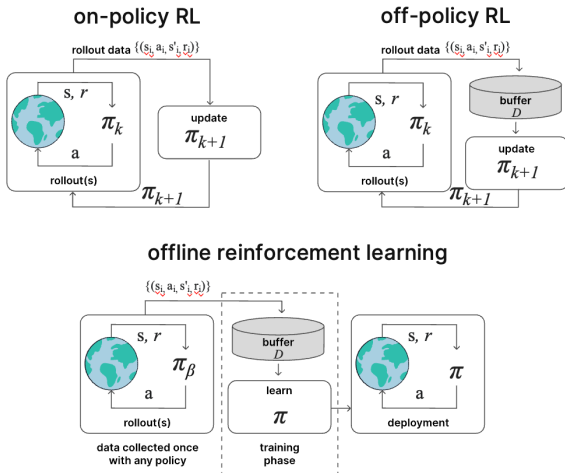
1 Off-Policy RL:

Собираем отложенный опыт, на котором учимся (DQN, DDPG, SAC)

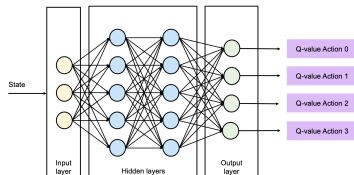
2 On-Policy RL:

Обучаем стратегию по мере исследования среды (**Actor-Critic**, PPO)

3 Offline RL: Учимся на оффлайн датасете экспертных данных (обычный sequence modeling)



ЧТО ИЗУЧИЛИ НА ПРОШЛОЙ ЛЕКЦИИ



- Зная, что функция $Q(s, a)$ определяет полезность действия a в состоянии s

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s, a_0 = a \right]$$

- Мы занимались приближением $\hat{Q}(s, a, \theta) \approx Q(s, a)$ с помощью нейросети
- Для выбора действия в среде, мы выбирали $a = \operatorname{argmax}_a Q(s, a)$

Недостатки Value-based методов (DQN)

Value-based подходы (например, DQN): оценивают функцию ценности $Q(s, a)$, а действия выбираются как

$$a = \arg \max_a Q(s, a)$$

Основные недостатки:

- Плохо масштабируются на непрерывные пространства действий
 - $\arg \max$ по непрерывному a — трудно вычислить
- Выбор действия не является дифференцируемой операцией
 - Нельзя напрямую обучать стратегию градиентом
- Отдельная политика: не явно задаётся, а косвенно через $Q(s, a)$
 - Это усложняет контроль над поведением агента
- Может страдать от переоценки Q-значений (overestimation bias)
- Исследование (exploration) ограничено ϵ -жадной стратегией
 - Это грубый механизм, особенно в сложных задачах

Определения:

- **Стохастическая стратегия** $\pi(a|s)$ — задаёт распределение вероятностей действий в состоянии s .

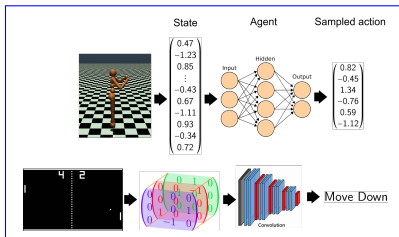
$$\pi(a|s) = \mathbb{P}[a_t = a \mid s_t = s]$$

- **Детерминированная стратегия** $\mu(s)$ — отображает каждое состояние в конкретное действие.

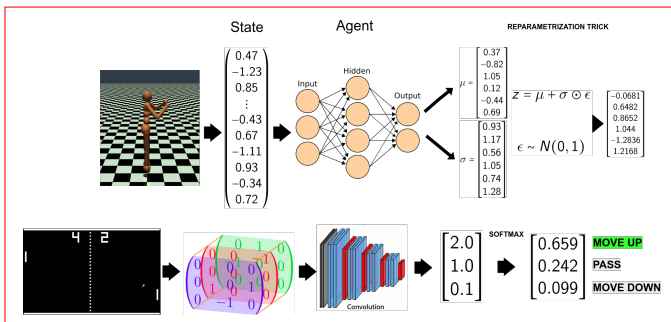
$$a = \mu(s)$$

- В общем случае, стохастическую стратегию можно свести к детерминированной, если

$$\mathbb{P}[a_t = a \mid s_t = s] \rightarrow 1$$



Детерминированная



Стохастическая стратегия

Переход к новой задаче оптимизации

- **Задача:** Обучить стратегию так, что бы вероятностное распределение над действиями максимизировало мат.ожидание награды

$$J = \mathbb{E}_{s_0 \sim p, a_0 \sim \pi(a|s_0), s_1 \sim p(s_0|a_0), a_1 \sim \pi(a|s_1)} [R(s, a, s')] \quad \text{где } a \sim \pi_\theta(s)$$

В случае одношаговой среды:

$$J = \mathbb{E}_{s_0 \sim p, a_0 \sim \pi(a|s_0)} [R(s, a)] = \int_s p(s) \int_a \pi_\theta(a|s) R(s, a) da ds$$

Что мы хотим уметь:

- 1 Уметь дифференцировать J по весам сети θ
- 2 Численно оценить ∇J (засемплировать градиент)

Проблема: просто продифференцировав по θ мы потеряем наше вероятностное распределение, а значит решив проблему с дифференцированием, мы обретём проблему с численной оценкой (не сможем засемплировать эту оценку)

$$J = \mathbb{E}[R(s, a)] = \int_s p(s) \int_a \nabla \pi_\theta(a|s) R(s, a) da ds$$

Решение: воспользуемся математическим приёмом

$$\nabla \log \pi(z) = \frac{1}{\pi(z)} \nabla \pi(z) \quad \text{тогда:} \quad \pi(z) \nabla \log \pi(z) = \nabla \pi(z)$$

Выведем пошагово итоговую оценку градиента нашей функции

$$\nabla J = \int_s p(s) \int_a \nabla \pi_\theta(a|s) R(s, a) da ds$$

$$\nabla J = \int_s p(s) \int_a \pi_\theta(a|s) \nabla \log \pi_\theta(a|s) R(s, a) da ds$$

Заметим, что у нас осталось матожидание

$$\nabla J = \underbrace{\int_s p(s) \int_a \pi_\theta(a|s) \nabla \log \pi_\theta(a|s) R(s, a) da ds}_{\text{мат. ожидание}}$$

$$\nabla J = \mathbb{E}_{s,a}[\nabla \log \pi_\theta(a|s) R(s, a)]$$

Засемплируем его и получим итоговый objective

$$\nabla J \approx \frac{1}{N} \sum_{t=0}^N \nabla \log \pi(a_t|s_t) R(s_t, a_t)$$

Интерпретация формулы: оптимизируем веса стратегии так, что бы её действием в состоянии s максимизировать **награду** $R(s, a)$

$$\nabla J \approx \frac{1}{N} \sum_{t=0}^N \nabla \log \pi(a|s) R(s, a)$$

Интерпретация формулы: оптимизируем веса стратегии так, что бы её действием в состоянии s максимизировать **будущую награду до конца эпизода** $Q(s, a)$

$$\nabla J \approx \frac{1}{N} \sum_{t=0}^N \nabla \log \pi(a|s) Q(s, a)$$

$$\nabla J \approx \frac{1}{N} \sum_{t=0}^N \nabla \log \pi(a|s) Q(s, a)$$

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} J$$

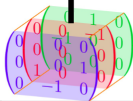
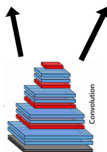
$$L_{critic} = (Q_{\theta}(s_t, a_t) - r_t + \gamma Q_{\theta}(s_{t+1}, a_{t+1}))^2$$

$$\theta^{new} = \theta^{old} - \alpha \nabla_{\theta} L_{critic}(\theta^{old})$$

MOVE UP
PASS
MOVE DOWN

$\begin{bmatrix} 0.659 \\ 0.242 \\ 0.099 \end{bmatrix}$

$Q_{\theta}(s_t, a_t)$



Actor-Critic

1. Генерируем данные взаимодействия со средой (Rollout):

- Внутри эпизода :
 - Наблюдаем s_t
 - Выбираем $a_t \sim \pi_\theta(a_t|s_t)$
 - Получаем r_t, s_{t+1}
 - Оцениваем $Q(s_t, a_t)$ через критика

2. Обновление параметров актора:

$$\theta \leftarrow \theta + \alpha \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot Q(s_t, a_t)$$

3. Считаем loss критика:

$$L_{critic} = (Q_\theta(s_t, a_t) - (r_t + \gamma Q_\theta(s_{t+1}, a_{t+1})))^2$$

4. Обновляем веса критика:

$$\theta^{new} = \theta^{old} - \alpha \nabla_\theta L_{critic}(\theta^{old})$$

Actor-Critic

Преимущества:

- **Поддержка непрерывных действий:** в отличие от DQN, Actor-Critic работает и в непрерывном пространстве действий, где невозможно использовать $\arg \max_a Q(s, a)$.
- **Онлайн-обновление:** обновление происходит после каждого шага, нет необходимости ждать заполнения replay buffer -> меньшие требования к железу.

Недостатки:

- **Шумный градиент:** стохастическая политика и оценки критика могут вносить шум в обновления.
- **Нестабильность:** совместное обучение актора и критика может приводить к расходимости, особенно если один из них сильно отстаёт.
- **Отсутствие replay buffer (в on-policy вариантах):** делает обучение менее sample-efficient по сравнению с off-policy методами как DQN.

Advantage Actor-Critic

1. Advantage функция показывает преимущество определённого действия a в состоянии s :

$$A(s, a) = Q(s, a) - V(s)$$

2. Меньшая дисперсия градиента (при использовании Advantage):

$$\nabla_{\theta} J(\theta) = \mathbb{E}[\nabla_{\theta} \log \pi_{\theta}(a|s) \cdot A(s, a)]$$

3. Зная, как вывести Q через V , мы знаем как оценить $A(s, a)$:

$$\text{Если } Q(s, a) = r + \gamma V(s') \text{ то } A(s, a) \approx r + \gamma V(s') - V(s)$$

3. Advantage Actor Critic (A2C) :

$$\nabla_{\theta} J(\theta) = \mathbb{E}[\nabla_{\theta} \log \pi_{\theta}(a|s) \cdot r + \gamma V(s') - V(s)]$$

Преимущества и недостатки Advantage Actor-Critic (A2C)

Преимущества:

- **Сниженная дисперсия градиента** Использование advantage-функции помогает уменьшить шум
- **Синхронное обучение на нескольких средах:** Параллельные агенты взаимодействуют с копиями среды и отправляют градиенты централизованно — повышает стабильность и сходимость.
- **Онлайн и sample-efficient:** каждый rollout сразу используется для обучения, без replay buffer.
- **Гибкость:** работает с непрерывными и дискретными действиями.

Недостатки:

- **Он-policy ограничение:** нельзя переиспользовать старые данные, как в DQN или DDPG.
- **Зависимость от качества критика:** плохая оценка $V(s)$ даёт плохой advantage, что вредит актору.
- **Чувствительность к гиперпараметрам:** особенно к длине rollout, шагу обучения и масштабам наград.